

# Reviewer Pack — Minimal Local Index of Graphs and Disjointness Communication...

Entry 5260b2ba8426 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 06:42:36 UTC

## Minimal Local Index of Graphs and Disjointness Communication Complexity

Entry ID: 5260b2ba8426 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 06:42:36 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic Topology: Local Indices **Field B** (complexity object): Communication Complexity: Disjointness

#### Statement:

{‘sentence\_1’: ‘The minimal local index of a graph  $G$ , defined as the minimum number of vertices required to cover all edges with open sets in a given neighborhood, lower bounds the randomized communication complexity of the disjointness problem on  $G$ .’, ‘sentence\_2’: ‘For all graphs  $G$  with  $n$  vertices, the local index of  $G$ , denoted by  $\text{loc\_idx}(G)$ , satisfies  $\text{loc\_idx}(G) \geq \Omega(n^{1/3})$  for the disjointness problem.’, ‘sentence\_3’: ‘This lower bound holds even when considering any fixed constant additive error.’}

#### Rationale (proposer’s reasoning):

{‘sentence\_1’: ‘The local index of a graph captures the geometric complexity of its structure, which is expected to be related to the communication complexity of problems defined on the graph.’, ‘sentence\_2’: ‘Disjointness being a fundamental problem in communication complexity, we expect that the structural complexity of the graph influencing the disjointness process directly relates to the communication complexity.’, ‘sentence\_3’: ‘By linking these two areas, this conjecture may provide new insights into the nature of communication problems and their inherent complexities.’}

**Taxonomy category:** COMM\_DISJ (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 0e126f3dd773de6d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture will be supported if the local index of all generated graphs meets or exceeds the lower bound of  $n^{1/3}$  with at least 80% of the computed randomized communication complexities being within a constant factor of the expected value, and no seed produces a communication complexity outside the expected range. The conjecture will

be falsified if less than 80% of the communication complexities are within this range or if any seed yields a communication complexity greater than the expect

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | HITS             | SAFE             |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | UNCERTAIN | 1.00       | HITS             | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 5 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "minimal local index" AND "communication complexity" AND graph - "algebraic topology" AND "local indices" AND disjointness - "randomized communication complexity" AND  $\Omega(n^{(1/3)})$  AND open sets

**Top relevant hits considered:** - [<http://arxiv.org/abs/1201.1666v1>] A direct product theorem for bounded-round public-coin randomized communication complexity - [<http://arxiv.org/abs/1703.07521v5>] Lifting randomized query complexity to randomized communication complexity - [<http://arxiv.org/abs/2104.1127>] The Randomized Communication Complexity of Randomized Auctions - [s2:10.1016/S0140-6736(18)30994-2] Measuring performance on the Healthcare Access and Quality Index for 195 countries and territories and selected subnatio - [s2:10.1145/2307636.2307674] WIISARD: a measurement study of network properties and protocol reliability during an emergency response

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90s$  wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math

def generate_random_graph(n):
    graph = [[0] * n for _ in range(n)]
    edges = set()
    while len(edges) < (n * (n - 1)) // 2:
        i, j = random.sample(range(n), 2)
        if i > j:
            i, j = j, i
        if (i, j) not in edges:
            graph[i][j] = 1
            graph[j][i] = 1
            edges.add((i, j))
    return graph

def local_index(graph):
    n = len(graph)
    covered_edges = set()
```

```

for i in range(n):
    for j in range(i + 1, n):
        if graph[i][j] == 1:
            covered_edges.update([(k, i), (k, j)] for k in range(n) if graph[k][i] == 1 or graph[k][j] == 1)
return len(covered_edges)

def communication_complexity(graph):
    n = len(graph)
    # Simplified version of the communication complexity calculation
    return n * (n - 1) // 2

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    graph = generate_random_graph(n)
    loc_idx = local_index(graph)
    comm_complexity = communication_complexity(graph)

    expected_comm_complexity = math.ceil(n ** (1/3))
    if comm_complexity < expected_comm_complexity:
        return {
            "metric_name": "communication_complexity",
            "metric_value": comm_complexity,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"Graph with n={n} has loc_idx={loc_idx}, but communication complexity is {comm_complexity}"
        }

    return {
        "metric_name": "communication_complexity",
        "metric_value": comm_complexity,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{"seed": {seed}, "metric_name": "{result['metric_name']}"}, {"metric_value": {result['metric_value']}"})
        results.append(result)

    if all(result["conjecture_holds"] for result in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    mean_comm_complexity = sum(result["metric_value"] for result in results) / len(results)
    std_comm_complexity = math.sqrt(sum((result["metric_value"] - mean_comm_complexity) ** 2 for result in results) / len(results))

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_comm_complexity} std={std_comm_complexity} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        print(f"RESULT: NOT SUPPORTED")

```

```

    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture"])
    print(f"RESULT: FALSIFIED counterexample=\"communication_complexity < expected\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_91b5dc67.py", line 75, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_91b5dc67.py", line 49, in run_trial
    loc_idx = local_index(graph)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_91b5dc67.py", line 37, in local_index
    covered_edges.update([(k, i), (k, j)] for k in range(n) if graph[k][i] == 1 or graph[k][j] == 1)
    ~~~~~
TypeError: unhashable type: 'list'

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified according to the pre-registered criteria. | next: Investigate and fix the error in the test code to allow for proper evaluation of the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 15907        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 10030        |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 9365         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 14969        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14624        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10401        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10041        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10610        |
| 9 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 11778        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 107726 ms total latency. Provider mix: {'ollama\_remote': 9}

*(full prompt+response transcripts available in research/audit/5260b2ba8426.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/5260b2ba8426.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/5260b2ba8426.tar.gz (if generated)